100 Days of Linux - Day 070

Title: Practical Assignment - Bash Shell Scripting

Today's Goal
Apply everything learned in the Shell Scripting module by building realistic Bash automation workflows. No new commands are introduced today.

Why This Matters
Shell scripting is widely used to automate deployments, backups, monitoring, ETL jobs, and server administration. Today's assignment combines all scripting concepts into one practical exercise.

Topics Covered
Shebang, chmod, variables, read, command-line arguments, if/elif/else, for loops, while loops, functions

Scenario
Create a script named **project_setup.sh** that prepares a new Linux project by collecting user information, validating input, displaying a summary, and automating repetitive tasks.

Practical Assignment (Exactly 5 Tasks)

1. Create project_setup.sh with a proper shebang (#!/bin/bash). Add greet() and summary() functions, make the script executable, and run it.

2. Ask the user for Project Name, Owner Name, and Environment (Dev/Test/Prod) using the read command. Store the values in variables.

3. Accept an optional command-line argument for the project version. If no argument is provided, display 'Version: 1.0 (Default)'.

4. Use if/elif/else to validate the environment (Dev, Test, or Prod). Then use a for loop to create and display the folders src, logs, config, and backup. Finally, use a while loop to print setup steps 1 through 5.

5. Display a formatted project summary using the summary() function, save the summary to project_report.txt, execute the script successfully, and display your current working directory.

Hints
Build and test one section at a time. Verify variables, then conditions, then loops, and finally functions.

Mini Challenge
Extend project_setup.sh by adding a menu-driven interface that lets the user create another project, view the last project summary, or exit.

Common Mistakes
Forgetting to make the script executable, missing the closing 'fi'

or 'done', or using spaces around '=' when assigning variables.

Did You Know?
Many CI/CD pipelines begin by running Bash scripts that prepare directories, validate configuration, and launch application deployments.

Pro Tip
Organize large scripts into small functions such as greet(), collect_input(), validate_environment(), create_structure(), and summary().

Answer Key (Instructor Only)
Q1: Create script, chmod +x project_setup.sh, ./project_setup.sh
Q2: Use read PROJECT, read OWNER, read ENVIRONMENT
Q3: Use $1 with an if statement to handle the default version
Q4: Validate ENVIRONMENT with if/elif/else, create folders in a for loop, count 1-5 with a while loop
Q5: Call summary(), redirect output to project_report.txt, run pwd

Homework
Enhance project_setup.sh to check whether the project directory already exists before creating it, and prompt the user before overwriting existing files.

Next Day Preview
Tomorrow you'll begin the final stretch of the course with more advanced Linux automation, revision, and capstone-style practical exercises leading to the 100-day completion.